Strong Method Problem Solving

(Topic 7)





Course Contents

Again..Selected topics for our course. Covering all of AI is impossible!

Key topics include:

- Introduction to Artificial Intelligence (AI)
- Knowledge Representation and Search
- Introduction to AI Programming
- Problem Solving Using Search
- Exhaustive Search Algorithm
- Heuristic Search
- Techniques and Mechanisms of Search Algorithm
- Knowledge Representation Issues and Concepts
- Strong Method Problem Solving
- Soft Computing and Machine Learning

Strong Method Problem Solving (knowledge-intensive or strong method)

- Introduction to expert system
  - Rule-based expert system
- Case-based reasoner
- Model-based
- Hybrid expert system

Introduction to Expert system

- Uses knowledge specific to a problem domain
- With the help of human domain experts
- System emulates the expert's methodology and performance
- Tend to be specialist, focusing on narrow set of problems; theoretical and practical
- Human expert provide the knowledge augmented theoretical understanding of the problem domain with tricks, shortcuts and heuristics for using the knowledge gained through problem-solving experience

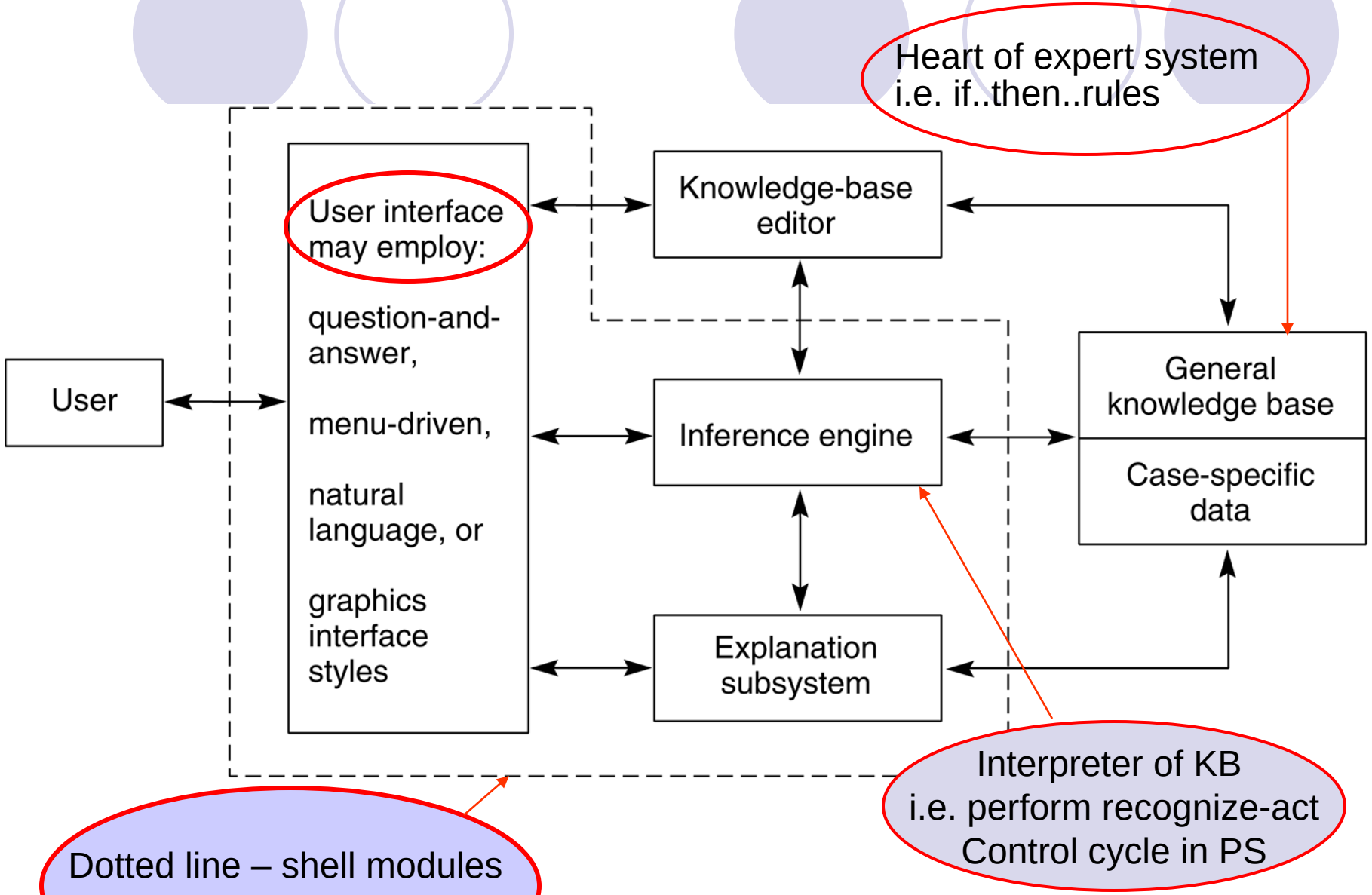
Introduction to Expert system

- Because of their heuristic, knowledge-intensive nature, expert systems generally:
  - **Support inspection of reasoning processes**- provide information and explanations of choices and decisions made by the program
  - **Allow easy modification in adding and deleting skills from knowledge base**- programs are easily prototyped, tested and changed. Modifying a rule has no global syntactic side effect
  - **Reason heuristically, using imperfect knowledge to get useful solutions** – tricks of the trade and rules of thumb, shortcuts

Introduction to Expert system

- Waterman(1986), expert system solves wide range of problems including:
  - Interpretation
  - Prediction
  - Diagnosis
  - Design
  - Planning
  - Monitoring
  - Instructions
  - control

Architecture of a typical expert system for a particular problem domain.



Why is the separation of KB and Inference engine necessary?

1. Makes it possible to represent knowledge naturally, humanly rather than computer code
2. ES builders can focus on capturing and organizing problem-solving knowledge rather than its implementation
3. Allows changes to be done separately
4. Allows same control and interface for variety of systems except for KB and case-specific data are emptied for new application

Architecture of a typical expert system for a particular problem domain..cont

- Case-specific data : facts, conclusion, information relevant to case under construction
- Explanation subsytem : justification of system's conclusion, how ..why.. Queries
- Knowledge based editor : helps to locate and correct bugs, assist addition of new knowledge, correct syntax, check updated KB

Guidelines to determine whether a problem is appropriate for expert system solution:

1. The need for the solution justifies the cost and effort of building an expert system-save money, time, life etc.

2. Human expertise is not available in all situations where it is needed-save time, money in remote sites

3. The problem may be solved using symbolic reasoning.

4. The problem domain is well structured and does not require commonsense reasoning (which is difficult to automate)

5. The problem may not be solved using traditional computing methods.

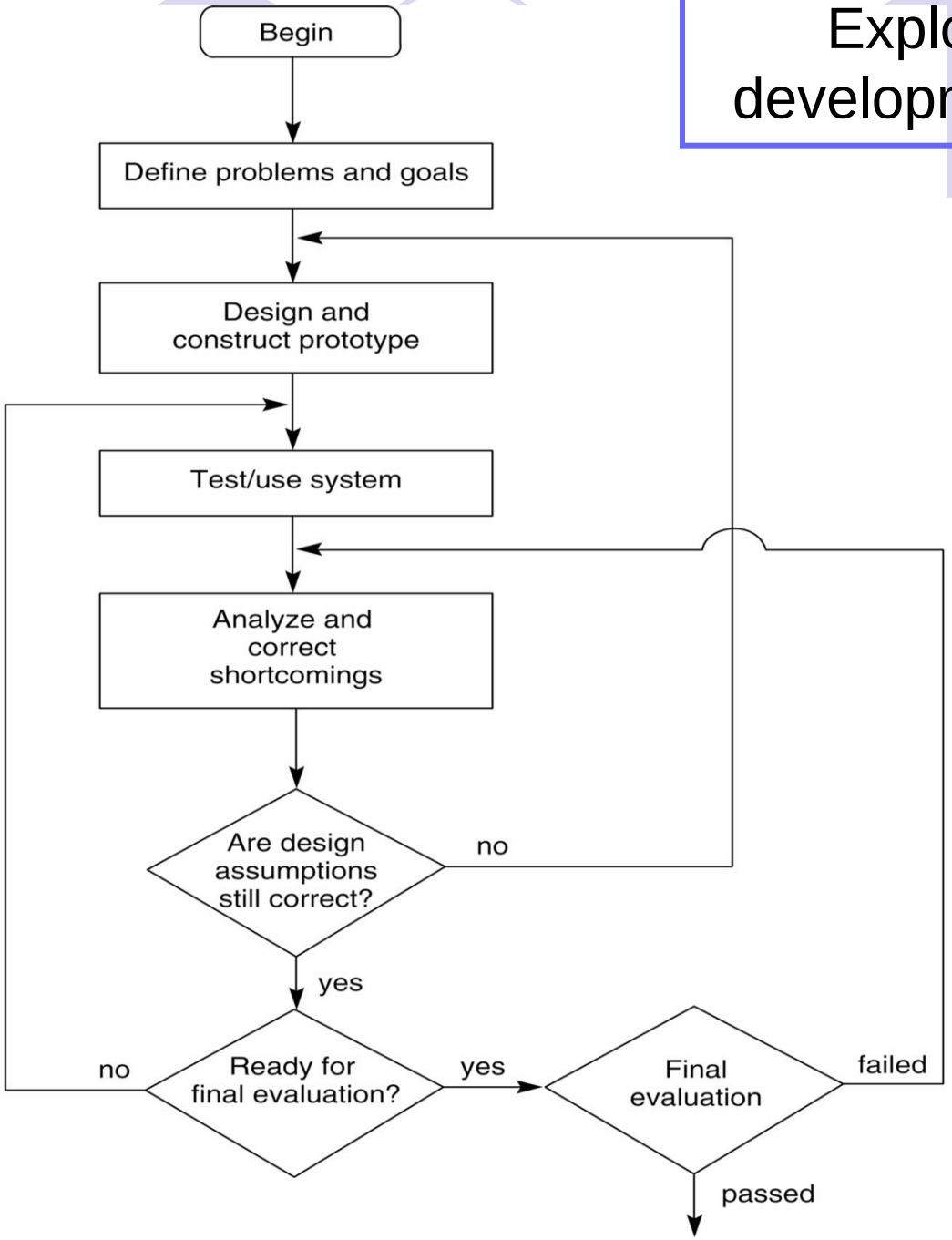
6. Cooperative and articulate experts exist – they are willing to share knowledge

7. The problem is of proper size and scope – not to capture ALL expertise

Who is involved in building ES?

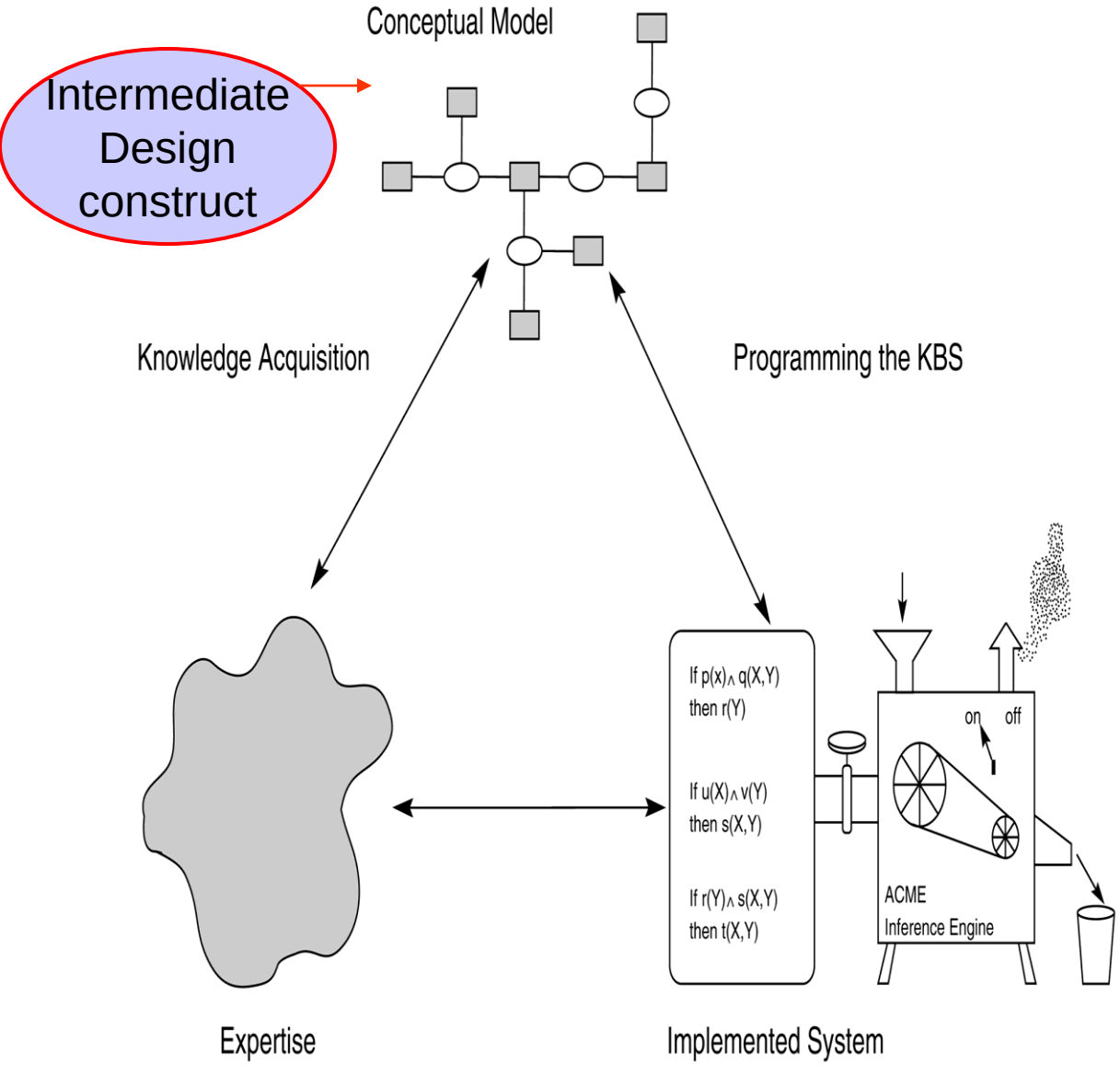
- **knowledge engineer**
  - AI language and representation expert
  - Select software/hardware tools
  - Help domain expert articulate necessary knowledge
  - Implement knowledge in correct/efficient KB
  - Ignorant of application domain
- **Domain expert**
  - Provide knowledge
  - Worked in the domain area
  - Understand its problem-solving techniques
  - Expert problem solver
  - Responsible to spell out skills to knowledge engineer
- **End user**
  - Determines major design constraint
  - Should be happy or else effort is wasted
  - Makes work quicker? Explanations? Correct information to system? Interface ok?

Exploratory development cycle



- a prototype is able to solve problems in small area of domain
- once prototype is implemented, knowledge engineer and domain expert test and refine its knowledge by giving it problem to solve and correcting its shortcoming
- if assumption correct, prototype incrementally extended until it becomes final system

The role of mental or conceptual models in problem solving.



- conceptual model
  - means knowledge engineer's evolving conception of domain knowledge
  - determines construction of formal KB

Rule-based Expert System: Production system and goal-driven prob solving

Rule 1: **if**
the engine is getting gas, and
the engine will turn over,
then
the problem is spark plugs.

Rule 2: **if**
the engine does not turn over, and
the lights do not come on
then
the problem is battery or cables.

Rule 3: **if**
the engine does not turn over, and
the lights do come on
then
the problem is the starter motor.

Rule 4: **if**
there is gas in the fuel tank, and
there is gas in the carburetor
then
the engine is getting gas.

A small expert system for **analysis of automotive problems-goal driven rule chaining**

- rule-based ES represent prob-solving knowledge as if… then…. Rules
- understood as production system model
- Goal-driven expert system- goal initially placed in working memory
- System matches rules conclusion with goal, select a rule and place premise in working memory

Figure 7.5: The production system at the start of a consultation in the car diagnostic example.

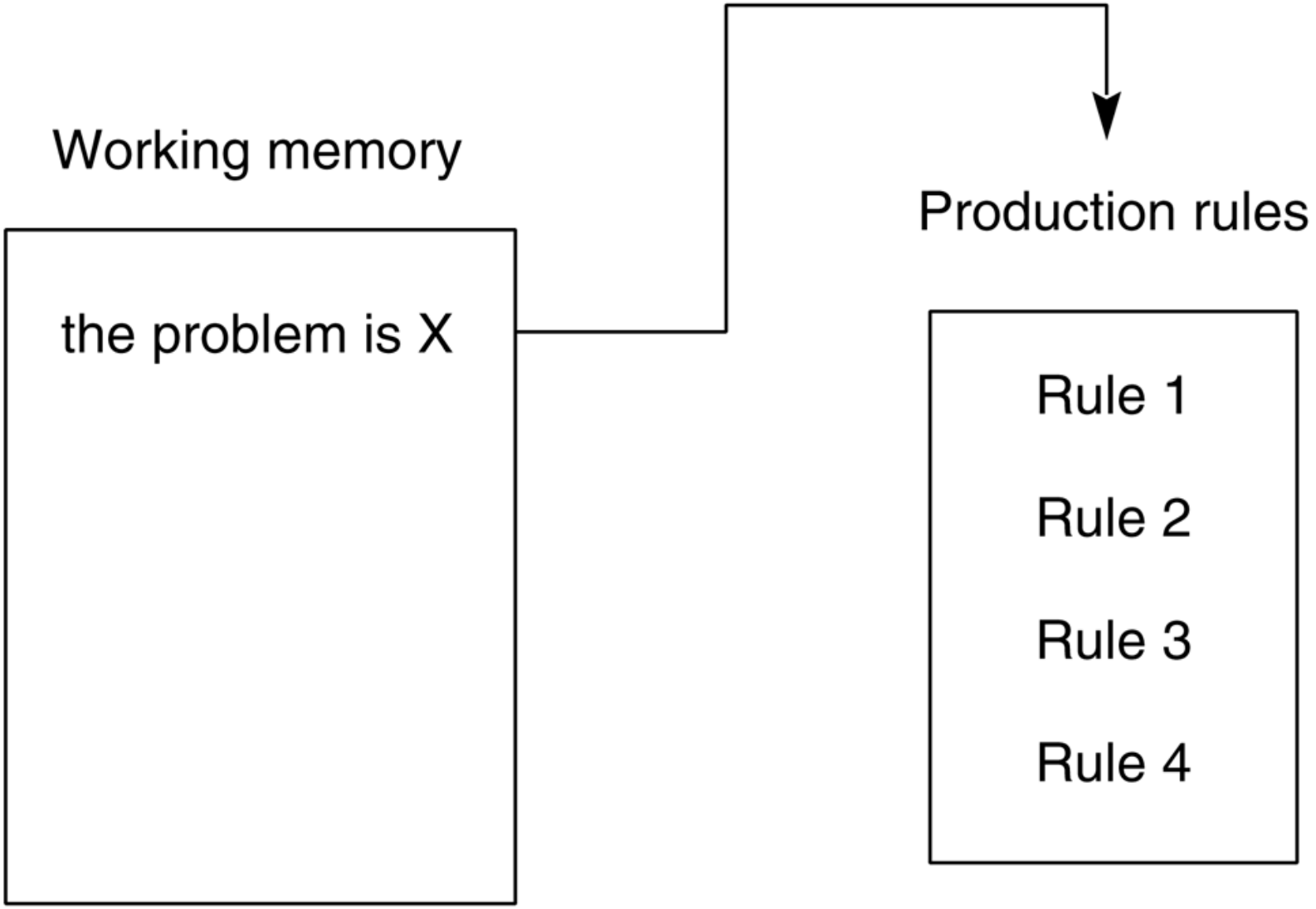


Figure 7.6: The production system after Rule 1 has fired.

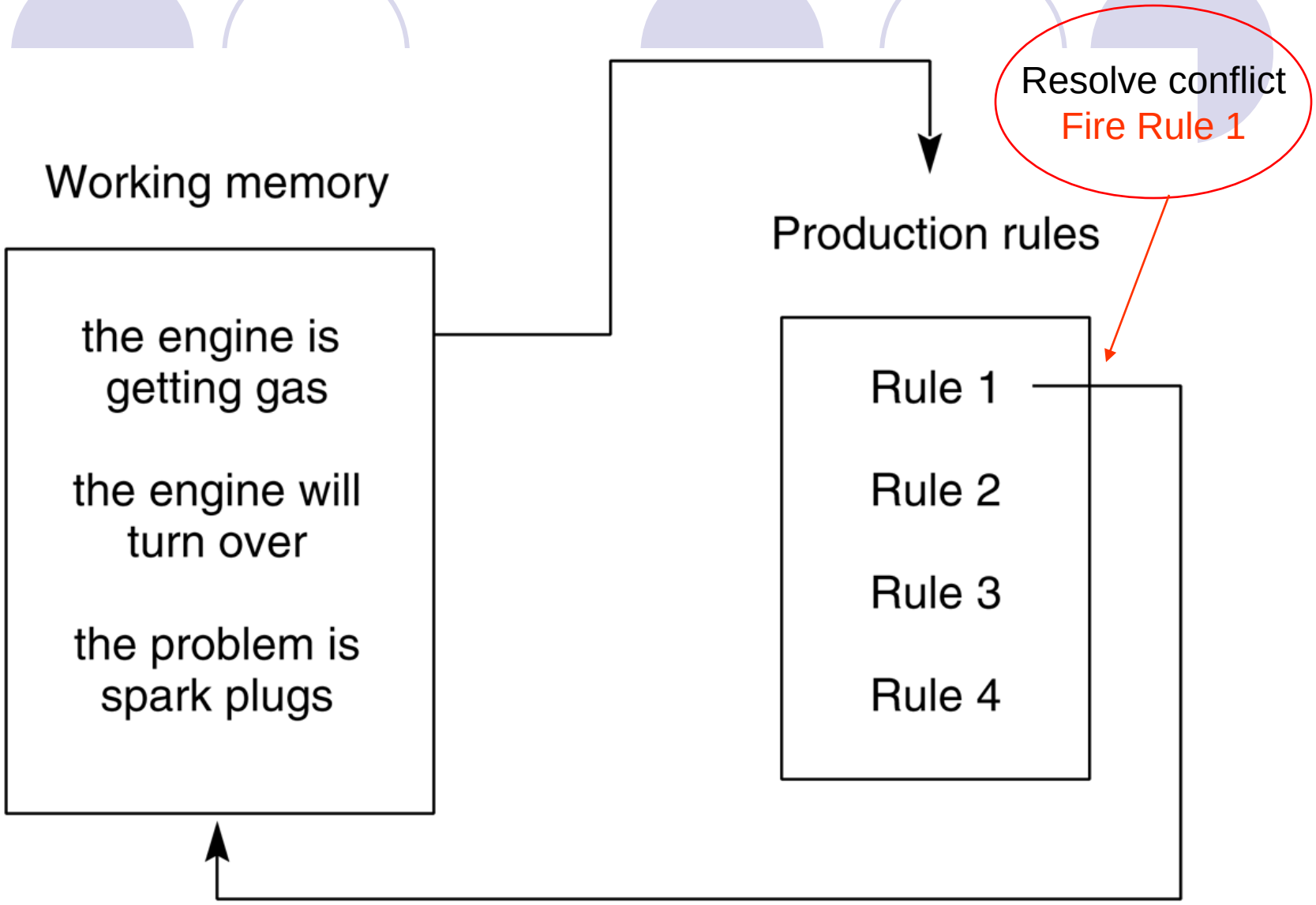
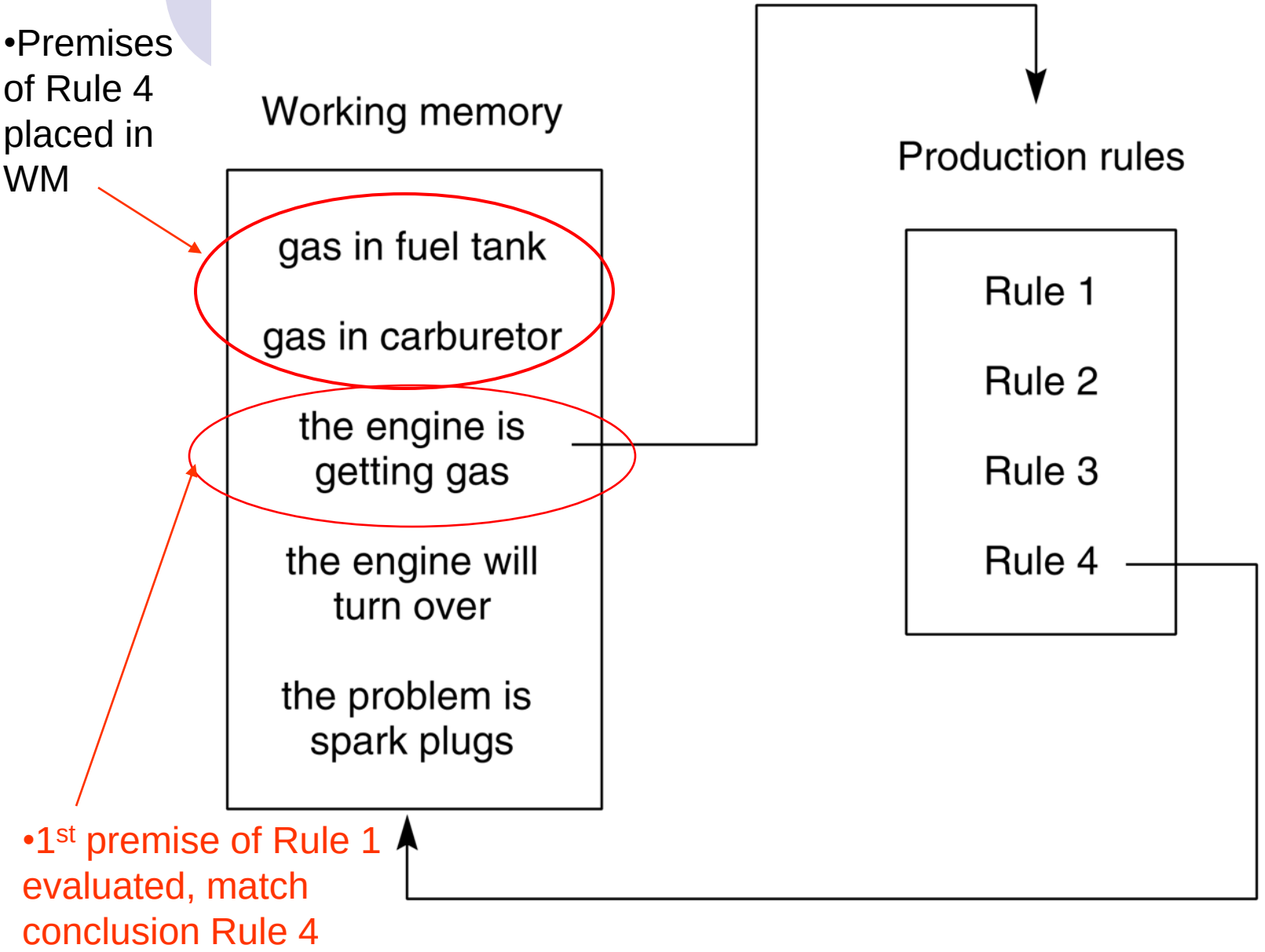
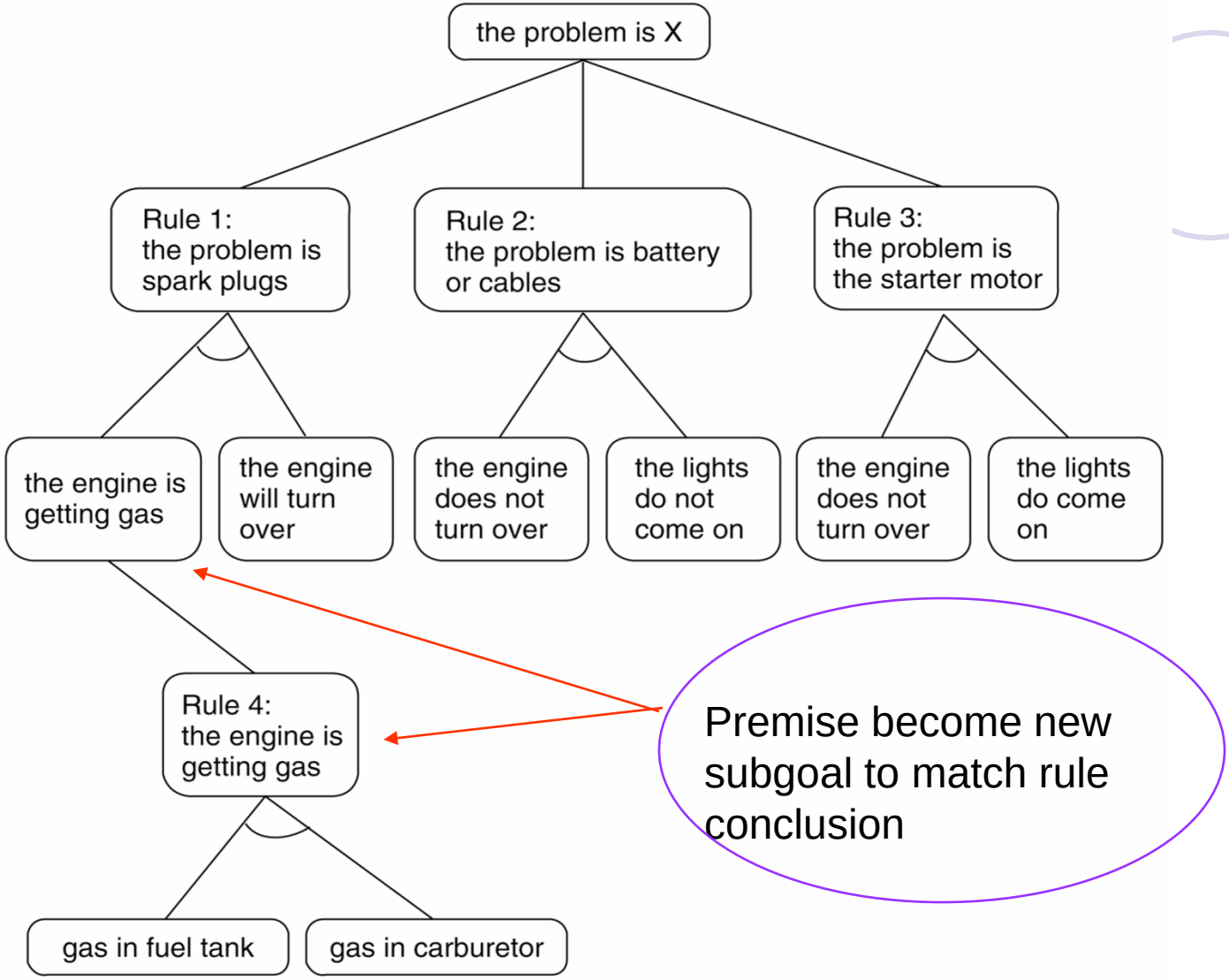


Figure 7.7: The system after Rule 4 has fired. Note the stack-based approach to goal reduction.





The and/or graph searched in the car diagnosis example, with the conclusion of Rule 4 matching the first premise of Rule 1.

- Query user about subgoals
- If all true,ES determine car doesn't start becoz of bad sparkplug

gas in fuel tank?
yes
gas in carburetor?
yes
engine will turn over?
why

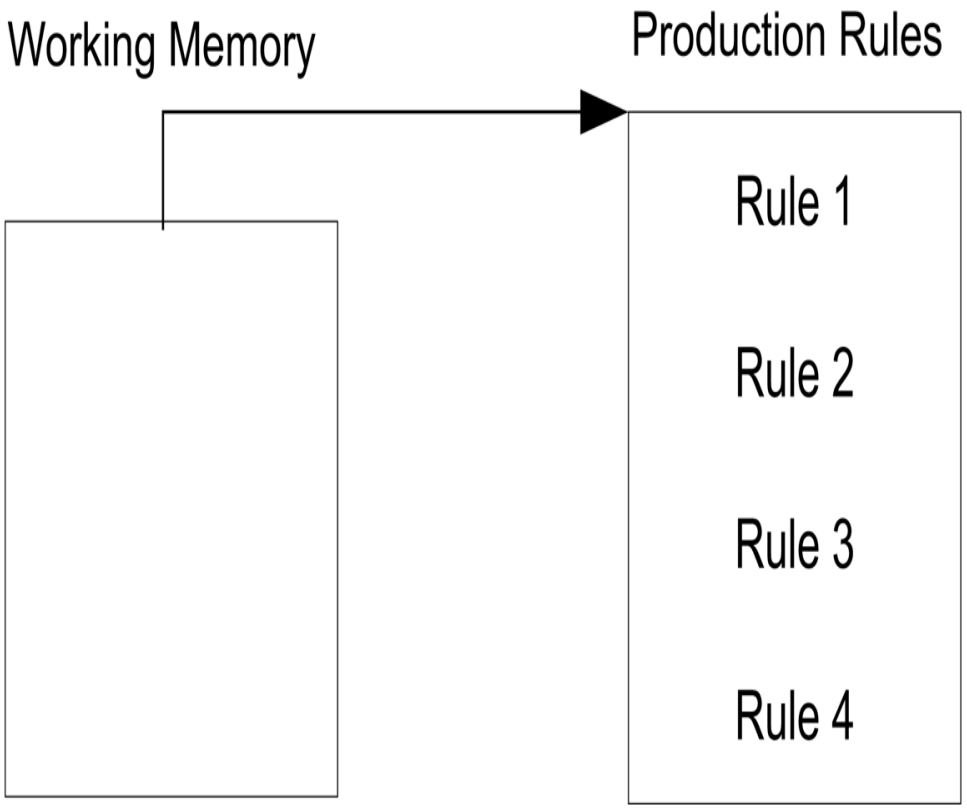
It has been established that:
1. the engine is getting gas,
therefore if
2. the engine will turn over,
then the problem is spark plugs.

how the engine is getting gas

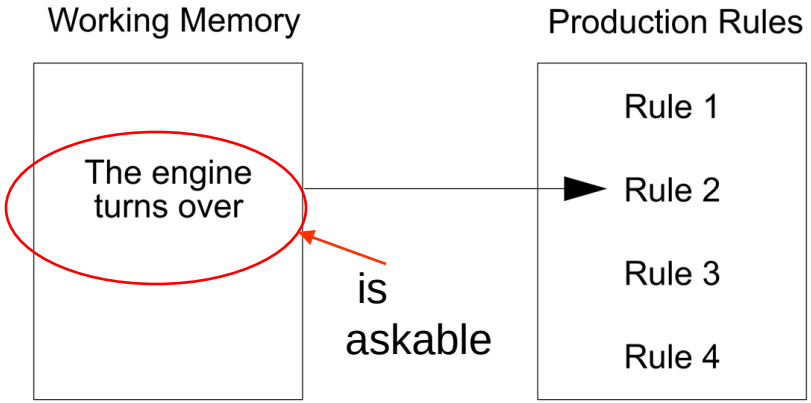
This follows from rule 4:
if
gas in fuel tank, and
gas in carburetor
then
engine is getting gas.
gas in fuel tank was given by the user
gas in carburetor was given by the user

The following dialogue begins with the computer asking the user about the goals present in working memory.

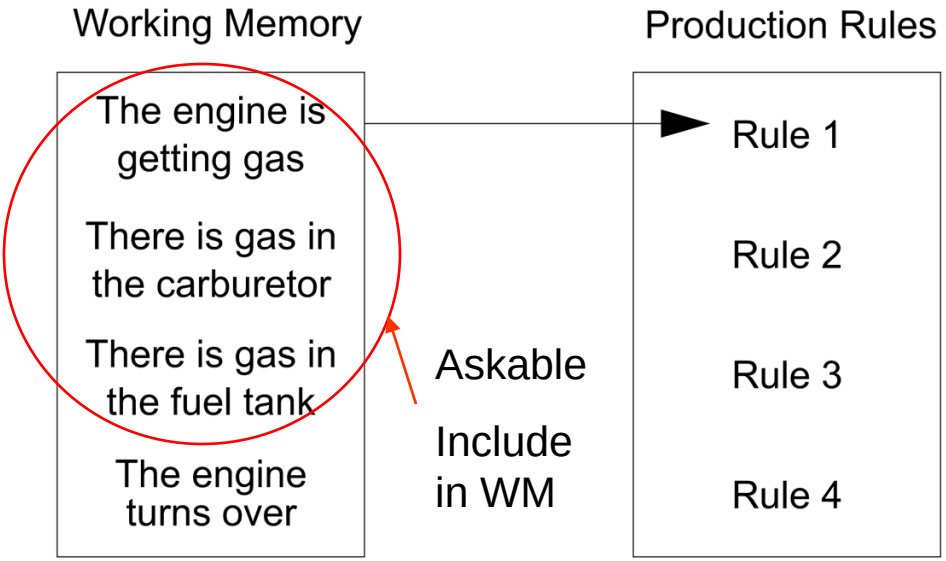
The production system at the start of a consultation for **data-driven reasoning**.



- breadth-first search is more common in data-driven resoning
- Compare WM with conditions of each rule
- Once all rules are considered,search starts again at beginning
- Examine rules – is info askable? If no, fail. Move to next rule



The production system after evaluating the first premise of Rule 2, which then fails



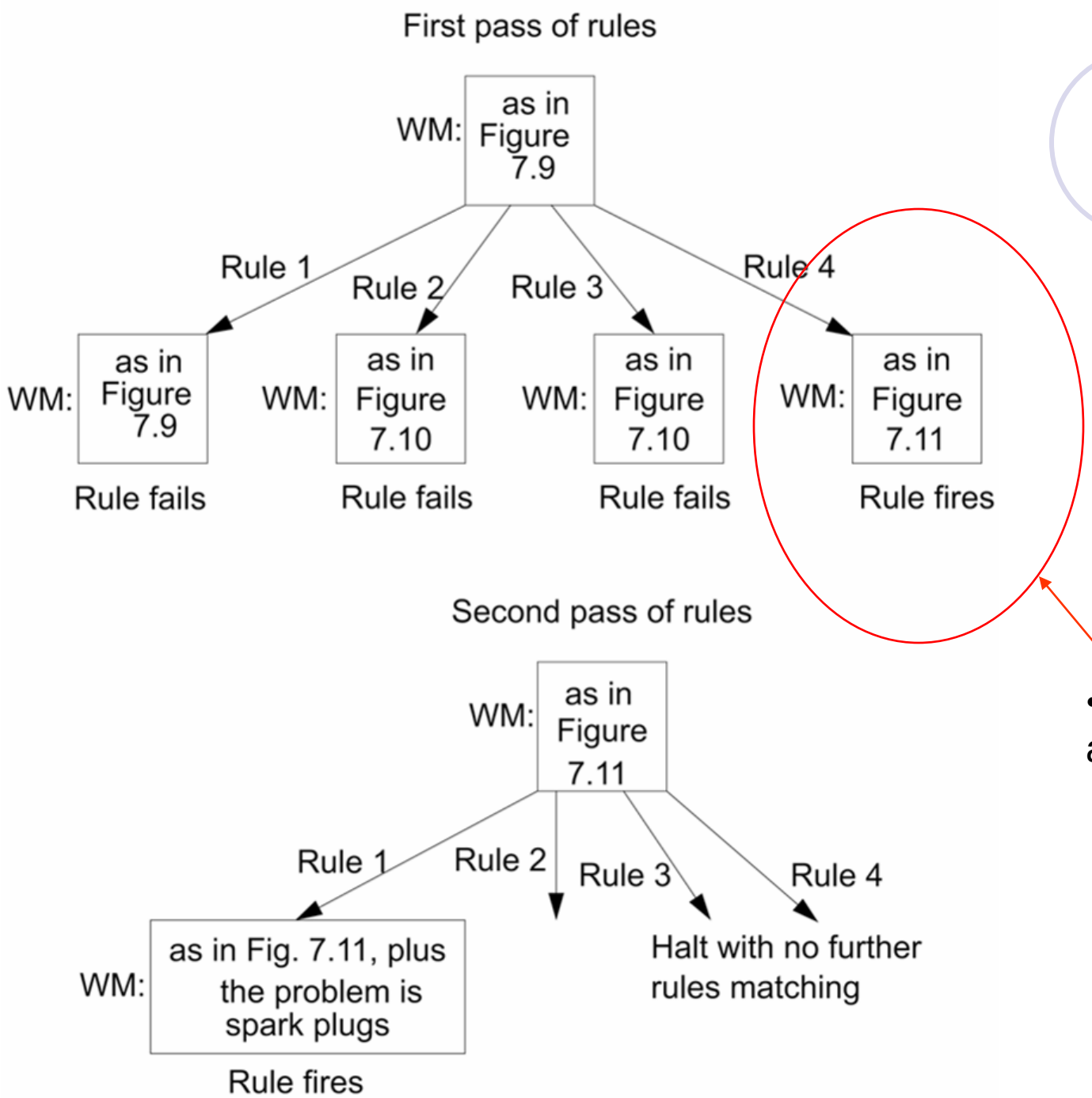
Rule 1: **if**
the engine is getting gas, and
the engine will turn over,
then
the problem is spark plugs.

Rule 2: **if**
askable { the engine does not turn over, and
the lights do not come on
then
the problem is battery or cables.

Rule 3: **if**
the engine does not turn over, and
the lights do come on
then
the problem is the starter motor.

Rule 4: **if**
askable { there is gas in the fuel tank, and
there is gas in the carburetor
then
the engine is getting gas.

The data-driven production system after considering Rule 4, beginning its second pass through the rules.



The search graph as described by the contents of working memory (WM) for the data-driven breadth-first search of the rule set of Section 7.2.1

•Premises are askable

Conclusion

- Data-driven ~ forward chaining search is diffused and less focused, goal orientation does not exist, search moves depending on rules order and discovery of new knowledge
- Goal-driven ~ backward chaining, goal is broken into subgoals, search is directed through the goal

Next..

- Model-based
- Case-based
- hybrid systems